

Output Heads as Inductive Bias

This is the second post in a series on neural-network inductive biases. The foundations post closed by naming two distinct axes at which the prior enters: the architecture (how the network computes features) and the output head (what the raw output *means*). The CNN post took the architecture axis on its first non-trivial step. This post takes the output-head axis, and the central claim is one sentence:

Every supervised neural network is a maximum-likelihood estimator under an assumed output distribution. The link function and the matching loss together specify that distribution, and the choice of distribution is itself an inductive bias about the data-generating process.

The foundations post quietly committed to three such distributions (Gaussian, Bernoulli, Categorical) without naming them as choices. This post names them, then extends the catalogue to counts (Poisson), to outputs with heteroscedastic noise (Gaussian with predicted variance), and to continuous proportions (sigmoid for bounded targets). Each is a different bet about what kind of object y is.

1. The unifying frame

Every supervised loss the foundations post used had the same shape:

$$L(\theta) = -\frac{1}{N} \sum_{n=1}^N \log p_{\theta}(y_n | x_n).$$

Minimizing the loss *is* maximum likelihood under an assumed conditional distribution $p_{\theta}(y | x)$. The network parameterizes that distribution; gradient descent moves the parameters toward higher likelihood of the observed data.

The output head is the bridge. The network produces a raw vector $z = f_{\theta}(x)$. A **link function** turns z into the natural parameter (or the mean, or a probability vector) of the assumed distribution. The **loss** is the negative log-likelihood of y under that distribution. The foundations post's organizing principle, *the network produces logits, the loss interprets them*, is exactly this: the link function and the assumed distribution together *are* the interpretation.

That is the whole story. The rest of the post is the unpacking.

2. The catalogue

Five canonical pairings, each one a member of the exponential family with its **canonical link** (the link that makes the math come out cleanest):

Output type	Link	Loss	Assumed distribution
Real-valued	identity	MSE	Gaussian (fixed variance)
Binary	logit (sigmoid)	BCE	Bernoulli
Categorical	softmax	cross-entropy	Categorical
Count	log (inverse exp)	Poisson NLL	Poisson
Positive continuous	reciprocal	Gamma NLL	Gamma

The first three are what the foundations post already built. The rest of the post extends the table. Each row is a different commitment about what kind of object y is. Real-valued and unbounded? Gaussian. A 0 or 1? Bernoulli. One of K mutually exclusive classes? Categorical. A non-negative integer event count? Poisson.

The link makes the network’s unconstrained raw output $z \in \mathbb{R}$ into a valid parameter for the distribution: a real mean for Gaussian, a probability in $[0, 1]$ for Bernoulli, a probability vector on the simplex for Categorical, a positive rate for Poisson.

3. The canonical link theorem

The foundations post observed that BCE and softmax cross-entropy both have gradient $\partial L / \partial z = \mathbf{p} - \mathbf{y}$ with respect to the logits. The foundations post called this “no coincidence” but did not prove the general statement. Here it is.

Write an exponential family in natural form:

$$p(y \mid \eta) = h(y) \exp(\eta \cdot T(y) - A(\eta)),$$

where η is the **natural parameter**, $T(y)$ is the **sufficient statistic**, and $A(\eta) = \log \int h(y) \exp(\eta \cdot T(y)) dy$ is the **log-partition function**. For Bernoulli, $\eta = \log(p/(1-p))$ is the logit; for Categorical, $\eta_i = \log p_i$ up to a shared shift; for Gaussian (fixed variance), $\eta = \mu/\sigma^2$; for Poisson, $\eta = \log \lambda$.

A foundational identity in exponential families: differentiating the normalization condition $\int p(y \mid \eta) dy = 1$ with respect to η yields

$$A'(\eta) = \mathbb{E}[T(y)].$$

The derivative of the log-partition function is the expected sufficient statistic. Call this $\hat{p}$, the model’s “prediction” in the relevant sense (for Bernoulli, $\hat{p}$ is the success probability; for Categorical, the probability vector; for Gaussian, the mean; for Poisson, the rate).

Now suppose the network’s output z is the natural parameter, $\eta = z$. This is the **canonical link** choice. The negative log-likelihood is

$$-\log p(y \mid z) = -\log h(y) - z \cdot T(y) + A(z),$$

and its gradient with respect to z is

$$\frac{\partial}{\partial z} [-\log p(y \mid z)] = -T(y) + A'(z) = \hat{p} - y,$$

reading y as $T(y)$ (the sufficient statistic; for Categorical, the one-hot of y).

Theorem. *For any exponential-family distribution with its canonical link, the gradient of the NLL with respect to the network’s output is the expected sufficient statistic minus the observed.*

The foundations post’s “BCE and softmax cross-entropy both have $\mathbf{p} - \mathbf{y}$ ” pattern is one instance of this theorem. MSE (identity + Gaussian) is another: $\partial L / \partial z = z - y$, which is again $\hat{p} - y$ with $\hat{p} = z$. Poisson (log link + Poisson NLL) will be a third in section 5. The same theorem.

A pedagogical aside: this also says the canonical link is a *natural* choice, not just a convenient one. The gradient is geometrically a residual, the gap between the model’s expected value and what was observed. Nothing about the form of the response variable enters.

4. The foundations post’s three pairings, named

Brief recaps; the math is already in the foundations post.

Identity + Gaussian + MSE. Assumed distribution: $y \mid x \sim \mathcal{N}(\mu(x), \sigma^2)$ with σ^2 fixed and absorbed into the constant. Link: identity (the network’s output z is μ). NLL (dropping constants): $\frac{1}{2}(y - z)^2$, mean squared error. Gradient: $z - y$. Inductive bias: errors are symmetric, additive, and Gaussian. If the true distribution is heavy-tailed or skewed, MSE pays for that with biased estimates.

Logit + Bernoulli + BCE. Assumed distribution: $y \mid x \sim \text{Bernoulli}(p(x))$, $y \in \{0, 1\}$. Link: logit, $\eta = \log(p/(1 - p)) = z$; the inverse is sigmoid, $p = \sigma(z)$. NLL: $-y \log p - (1 - y) \log(1 - p)$, computed stably from z . Gradient: $p - y$. Inductive bias: the response is binary; a probabilistic prediction is well calibrated when training converges.

Softmax + Categorical + cross-entropy. Assumed distribution: $y \mid x \sim \text{Cat}(\mathbf{p}(x))$, $y \in \{0, \dots, K - 1\}$. Link: softmax, $\mathbf{p} = \text{softmax}(\mathbf{z})$. NLL: $-\log p_y = \text{logsumexp}(\mathbf{z}) - z_y$ in the stable form. Gradient: $p_i - \mathbb{1}[i = y]$, i.e. $\mathbf{p} - \mathbf{y}$ with $\mathbf{y}$ the one-hot. Inductive bias: the classes are mutually exclusive and exhaustive; the model cannot represent “two classes at once,” a real structural prior.

Three different objects ($y \in \mathbb{R}$, $y \in \{0, 1\}$, $y \in \{0, \dots, K - 1\}$), three different distributions, three different links. One pattern: $\hat{p} - y$.

5. Count data: log link + Poisson NLL

The first genuinely new pairing.

Assumed distribution: $y \mid x \sim \text{Poisson}(\lambda(x))$, $y \in \{0, 1, 2, \dots\}$, non-negative integers. The PMF is $p(y \mid \lambda) = \lambda^y e^{-\lambda} / y!$.

Link: logarithm, $\eta = \log \lambda = z$, so $\lambda = e^z$. The exponential guarantees $\lambda > 0$ for any real z , without any clamping or projection.

NLL:

$$-\log p(y \mid \lambda) = \lambda - y \log \lambda + \log y! = e^z - yz + \log y!$$

The $\log y!$ term does not depend on z and drops out of the gradient. For computing the loss value during optimization, we omit it as well (this shifts the loss by a y -dependent constant but does not affect the optimum).

Gradient:

$$\frac{\partial L}{\partial z} = e^z - y = \lambda - y = \hat{p} - y.$$

The canonical-link theorem again, a third instance with no new derivation needed.

In scratchnn:

```
class PoissonNLLLoss(Loss):
    def value(self, logits, y):
        z = logits[0]
        return math.exp(z) - z * y

    def grad(self, logits, y):
        z = logits[0]
        return [math.exp(z) - y]

    def probs(self, logits):
        return [math.exp(logits[0])]
```

Ten lines, including docstring elision. Same hand-derived backward as the other losses; gradient check (`tests/test_gradients.py` analog) passes at 10^{-11} .

Worked experiment. A 1-D regression where the true rate is non-monotonic, e.g. $\lambda(x) = \max(0.1, 2 + 5 \sin(\pi x))$ for $x \in [0, 2]$. Sample $y_n \sim \text{Poisson}(\lambda(x_n))$. Train two networks of identical body ($1 \rightarrow 16 \rightarrow 1$ with Tanh hidden), one with identity + MSE and one with log + Poisson NLL. See `examples/poisson_regression.py` for the full setup.

Findings, honestly. On this benign synthetic dataset, both models fit the conditional mean reasonably well. Neither predicts negative rates on a fine grid of test inputs, because the Tanh activations smooth the network output enough to keep predictions in a sensible range. Test Poisson NLL is roughly comparable between the two heads (within 0.1 nats per sample, with MSE marginally better on some seeds).

This is honest pedagogy: with a benign target distribution, both heads work. What matters is the *structural* difference:

- The MSE model’s *minimum* predicted rate on the grid is 0.068. The Poisson model’s minimum is 0.310. The Poisson head has a smoother lower bound built in (it is asymptotically $e^z \rightarrow 0$ as $z \rightarrow -\infty$, but very slowly).
- The MSE model *could* predict any negative number if the training data were less benign; the Poisson model *cannot* by construction.
- On a count dataset where many observations are 0 (rare events), the MSE model genuinely fails: it pays MSE penalty for being slightly positive on average, and its predictions cross zero. The Poisson head never does.

The Poisson NLL also accounts for the Poisson property $\text{Var}(y) = \mathbb{E}(y) = \lambda$. MSE assumes constant variance, which is wrong for counts where small means have small variance and large means have large variance. On a homoscedastic test dataset, this matters less; on real count data it can matter a great deal.

The structural argument is the load-bearing one. Use the matching head because the matching head encodes the constraint, even when on a benign dataset the constraint would not have been violated anyway.

6. Uncertainty: heteroscedastic Gaussian NLL

The foundations post’s MSE loss assumed Gaussian errors with *constant* variance, absorbed into the loss constant. That is fine when the noise really is constant. When the noise is input-dependent, the model has no way to report it. A point prediction does not say “I am confident here” or “I am unsure here.” The user has to find that out the hard way.

Heteroscedastic Gaussian regression fixes this with a two-output head. The network outputs (z_μ, z_s) per example, and the loss interprets them as the parameters of a per-input Gaussian:

$$\mu = z_\mu, \quad \sigma = \text{softplus}(z_s).$$

Softplus, $\text{softplus}(z) = \log(1 + e^z)$, is the smooth analogue of the positive part. It guarantees $\sigma > 0$ for any $z_s \in \mathbb{R}$, without clamping; its derivative is the logistic sigmoid, $\text{softplus}'(z) = \sigma_{\text{logistic}}(z)$.

NLL (dropping the $\frac{1}{2} \log(2\pi)$ constant):

$$L = \frac{(y - \mu)^2}{2\sigma^2} + \log \sigma.$$

The first term is the familiar squared error, rescaled by the predicted *precision* $1/\sigma^2$. A confident prediction (small σ) gets a *larger* MSE-like gradient. The second term is a regularizer that prevents the model from blowing σ up to infinity to wash out the first term.

Gradients, hand-derived:

$$\frac{\partial L}{\partial \mu} = \frac{\mu - y}{\sigma^2}.$$

The MSE gradient, scaled by predicted precision.

$$\frac{\partial L}{\partial \sigma} = -\frac{(y - \mu)^2}{\sigma^3} + \frac{1}{\sigma}.$$

Zero when $(y - \mu)^2 = \sigma^2$, the maximum-likelihood point: the model picks σ equal to the realized residual.

Chain through softplus to get the gradient with respect to the network output z_s :

$$\frac{\partial L}{\partial z_s} = \frac{\partial L}{\partial \sigma} \cdot \sigma_{\text{logistic}}(z_s).$$

In scratchnn:

```
class GaussianNLLLoss(Loss):
    def value(self, logits, y):
        mu = logits[0]
        s = softplus(logits[1])
        return 0.5 * (y - mu) ** 2 / (s * s) + math.log(s)
```

```

def grad(self, logits, y):
    mu = logits[0]
    z_s = logits[1]
    s = softplus(z_s)
    dm_u = (mu - y) / (s * s)
    ds = -((y - mu) ** 2) / (s ** 3) + 1.0 / s
    return [dm_u, ds * sigmoid(z_s)]

def probs(self, logits):
    return [logits[0], softplus(logits[1])]

```

Fifteen lines. Same Loss interface as the others; the only novelty is that the head produces *two* numbers per example.

Worked experiment. A 1-D function with input-dependent noise, $y = \sin(x) + \varepsilon$ where $\varepsilon \sim \mathcal{N}(0, \sigma(x)^2)$ and $\sigma(x) = |x|/3 + 0.1$ over $x \in [0, 6]$. Noise is small on the left, growing to about $\sigma = 2$ on the right. See `examples/heteroscedastic.py`.

Two networks of identical body ($1 \rightarrow 16 \rightarrow 1$ for MSE, $1 \rightarrow 16 \rightarrow 2$ for heteroscedastic): one with MSE, one with Gaussian NLL.

Findings:

x	true μ	true σ	MSE prediction	het μ	het σ
0.5	0.479	0.267	0.568	0.635	0.234
1.5	0.997	0.600	1.059	1.114	0.525
2.5	0.598	0.933	0.371	0.584	0.821
3.5	-0.351	1.267	-0.356	-0.098	1.115
4.5	-0.978	1.600	-0.806	-0.650	1.392
5.5	-0.706	1.933	-1.044	-1.046	1.625

The heteroscedastic model recovers the input-dependent σ within 10-15% across the range. The MSE model has no σ at all; it only predicts the mean.

To put them on equal footing for held-out comparison: fit a single global σ for the MSE model post-hoc (the RMS training residual), then compute Gaussian NLL on the test set under each model's predicted distribution.

Model	Test Gaussian NLL
MSE model + global σ	0.745
Heteroscedastic head	0.509

The heteroscedastic head improves test NLL by 0.24 nats per sample. That is the calibration premium: predicting variance separately for each x is worth a quarter of a nat over the homoscedastic assumption on this dataset. The qualitative gain is bigger than the numerical one. The MSE-plus-global- σ model is over-confident on the noisy right side of the domain and under-confident on the clean left side; the heteroscedastic head is correct on both.

This is the cleanest demonstration in the post that the choice of head buys something tangible (calibrated, input-dependent uncertainty) that the standard head cannot provide.

7. Link and likelihood are independent choices

The catalogue in section 2 paired each likelihood with its canonical link. That is the *natural* pairing, the one that makes the gradient collapse to $\hat{p} - y$ and the optimization land cleanly. It is not the *only* pairing.

Suppose the target y is a continuous proportion: $y \in [0, 1]$, something like “fraction of pixels lit,” “vote share,” or “relative humidity.” Not a class label, a *number*. We want the prediction $\hat{p} \in (0, 1)$ as well.

The natural link to enforce $\hat{p} \in (0, 1)$ is the sigmoid: $\hat{p} = \sigma(z)$. We can pair sigmoid with two different likelihoods:

Sigmoid + MSE on the sigmoid output. Loss: $L = \frac{1}{2}(y - \sigma(z))^2$. Implicit likelihood: Gaussian noise in proportion space. Gradient: $(\sigma(z) - y) \cdot \sigma'(z)$, the chain rule. This works fine in the bulk of the domain but saturates near $z \rightarrow \pm\infty$ (where $\sigma'(z) \rightarrow 0$ and gradient flow stalls), making it hard for the model to push very near the boundaries.

Sigmoid + Beta NLL. Predict the mean $\mu = \sigma(z_\mu)$ and a “precision” $\nu = \text{softplus}(z_\nu)$, set $\alpha = \mu\nu$, $\beta = (1 - \mu)\nu$, and use the Beta NLL,

$$L = -(\alpha - 1) \log y - (\beta - 1) \log(1 - y) + \log B(\alpha, \beta).$$

This treats y as drawn from a Beta distribution. It is far more sensitive than the MSE choice near the boundaries (where Beta log-density is heavy-tailed in the right way for proportion data) and predicts a full posterior over $[0, 1]$ instead of a point.

Same link (sigmoid). Different likelihoods (Gaussian on the sigmoid output vs Beta directly). The choice changes what is assumed about the noise distribution, how the loss treats extreme values, and whether the head is one output or two.

This section breaks the table from section 2. The link function is not uniquely determined by the assumed distribution. There is a canonical pairing (the one the theorem gives you), but it is one choice among many. The skill is recognizing the choice.

(Beta NLL is left out of `scratchnn`; it requires `math.lgamma` which is in the standard library and easy enough to add. The pedagogical point is that you *can* pair sigmoid with Beta and the library does not need anything fundamentally new.)

8. Beyond unimodal: a pointer to mixture density networks

All of the heads above assumed $p(y | x)$ is unimodal. When it is not (a robot arm with two valid joint configurations for the same end-effector pose; an inverse problem with multiple solutions; the conditional next-frame distribution for a stochastic video model), a unimodal head collapses. A Gaussian head will predict the mean of the modes, which may lie in a region of *zero density* under the true distribution.

A **Mixture Density Network (MDN)** outputs the parameters of a K -component Gaussian mixture: K means, K scales, and K mixture weights (the latter through a softmax head). The NLL is

$$-\log \sum_{k=1}^K \pi_k(x) \mathcal{N}(y \mid \mu_k(x), \sigma_k(x)^2).$$

Visually: standard regression collapses to the midpoint between two modes; the MDN captures each mode with its own component, each weighted by its own probability.

Not implemented in `scratchnn`. Mentioned here the way the foundations post mentions autograd: the natural extension, the framework is the same (output head plus matching NLL), the implementation composes pieces the library already has (softplus for σ , softmax for π , logsumexp for stable computation of the mixture log-density) plus the hand-derived gradient through the mixture. A reader exercise.

9. Inductive bias, the parallel axis

Pull the post’s frame together with the foundations post’s.

The foundations post’s section on inductive biases named two axes:

- **Architecture:** a choice of how the network computes features. An MLP assumes every input is independent of every other input; a CNN assumes locality and translation equivariance; an RNN assumes time-translation equivariance; a Transformer assumes pairwise interactions through attention.
- **Output head:** a choice of link function and matching loss, encoding a prior about the distribution of y given the network’s raw output.

This post extended the second axis with three new heads (Poisson for counts, heteroscedastic Gaussian for input-dependent uncertainty, sigmoid + Beta for bounded continuous), and named the unifying frame (MLE under an assumed distribution; canonical link + matching loss as the natural choice from the exponential family).

The two axes are *independent and composable*. The choice of body and the choice of head are made separately and combined freely:

- A CNN with a Poisson head: pixel-wise photon-counting imaging.
- A Transformer with a Categorical head over a vocabulary: a language model.
- An MLP with a heteroscedastic Gaussian head: a regression model with calibrated, input-dependent uncertainty.
- A CNN with a Bernoulli head per pixel: a binary segmentation model.

The architecture is a prior about the *function* (how features compose). The output head is a prior about the *distribution* (what y is). They multiply: a matched head buys sample efficiency *on top of* a matched body, not instead of it.

A matched head means:

- **Sample efficiency.** Constraints encoded by the link (non-negativity via log, boundedness via sigmoid, simplex membership via softmax) are free; the network does not spend parameters on relearning them.

- **Calibrated uncertainty.** A two-output Gaussian head reports per-input σ . A categorical head reports a probability vector. Both are usable by downstream decisions.
- **Honesty about the data.** A Poisson head says “I am modeling counts.” That is a statement to the reader of the code, as much as a setting on the optimizer.

A mismatched head means:

- Wasted capacity discovering constraints the link could have given for free.
- Pathological predictions in low-density regions (negative counts, point estimates between modes, miscalibrated confidence).
- Misleading uncertainty (MSE gives none; a poorly calibrated softmax is a known failure mode).

The same shape as the architecture story from the foundations post. Architectural priors and output-head priors are parallel commitments, each evaluable on the same axes (does it match the data? does it improve sample efficiency? does it hurt when mismatched?).

10. Library additions

Concrete additions to `src/scratchnn/neural_net.py`. All hand-derived backward, all fitting the existing `Loss` interface (`value`, `grad`, `probs`). All standard-library only.

- **softplus(z)** (helper): $\log(1 + e^z)$ in the numerically stable form $\max(z, 0) + \log(1 + e^{-|z|})$. Used inside **GaussianNLLLoss** to map a real-valued output to a positive scale parameter. Derivative is the sigmoid.
- **MSELoss** (already present after the foundations post restructure): identity link, mean squared error. Single-output regression.
- **PoissonNLLLoss** (~10 lines): single logit, log link. $L = e^z - yz$ (omitting $\log y!$). $\partial L / \partial z = e^z - y$. `probs` returns e^z .
- **GaussianNLLLoss** (~15 lines): two logits, $\mu = z_\mu$, $\sigma = \text{softplus}(z_s)$. $L = \frac{(y-\mu)^2}{2\sigma^2} + \log \sigma$. Gradient derived in section 6. `probs` returns (μ, σ) .

Each new loss has a `gradient_check` case. The anchor network is a **Tanh** MLP per the CLAUDE.md convention; the strict 10^{-4} tolerance holds (the actual worst error is on the order of 10^{-10}).

Not in the library:

- **Beta NLL** for sigmoid + bounded regression. Requires `math.lgamma` (available in the standard library, fine to add). Left as a reader exercise because the pedagogical point of section 7 is that link and likelihood are independent, not that we need every likelihood in the library.
- **MDN** (mixture density network). Compose `softplus`, `softmax`, and `logsumexp`; hand-derive the mixture gradient. Left as a reader exercise.

11. Handoff

This post covered the output-head axis of inductive bias: the unifying frame (MLE under an assumed distribution), the canonical link theorem (why $\hat{p}-y$ keeps showing up), three review pairings, two new worked examples (Poisson and heteroscedastic Gaussian), one example of link/likelihood independence (sigmoid + Beta), an MDN pointer, the parallel-axes synthesis, and the library additions.

The architecture axis continues in the next post (CNN, already done) and the ones after it: a fixed-context language model (the simplest neural approach to sequence modeling), then RNNs (temporal recurrence as the arbitrary-history approximation), then the Transformer as the capstone on attention. Each architectural choice composes with any of the output heads in this post. A language model is just a Transformer body with a Categorical head over a vocabulary; image classification is a CNN body with a Categorical head over classes; image regression (depth, optical flow) is a CNN body with a Gaussian head; pixel-wise photon counting is a CNN body with a Poisson head. The two axes multiply.

The series closes with a post on reinforcement learning, which is the genuinely-different third learning paradigm beyond supervised. There the heads-as-bias frame still applies (a policy head is a softmax over actions; a value head is identity + MSE on returns), but the training-time signal is no longer a per-example label. The whole second half of the series cycles back to RL eventually; this post is the last one entirely inside the supervised frame.